

GUI Application Lifecycle and Resource Management

1. Course Objectives

Writing a runnable App is easy, but writing a **stable** and **memory-leak-free** App requires extra attention. In this lesson, we will delve into App lifecycle management.

2. Lifecycle States

An App goes through the following states during its execution:

1. **Load:** System starts, `AppManager` scans `app.py`, executes `__init__`. At this time, the App is in a dormant state, occupying minimal memory.
2. **Launch:** User clicks the icon, executes `launch_with_animation` -> `initialize`. At this time, UI resources are created, memory usage increases.
3. **Running:** User interacts with the interface.
4. **Suspend/Background:** In Yahboom K230's GUI, it currently mainly supports single-task foreground operation. The so-called "background" means being destroyed.
5. **Destroy:** User clicks the return button, App exits, releases resources.

3. Proper Resource Release

In `BaseApp`, although there is no explicit `destroy` method, when the `self.screen` created by `launch_with_animation` is deleted, all controls mounted on it will be automatically deleted.

However, there are some resources that LVGL **cannot manage**, which must be manually released:

3.1 Timers

If you created timers in your App:

```
self.timer = Timer(1)
self.timer.init(period=1000, mode=Timer.PERIODIC, callback=self.update)
```

You **must** close it when exiting, otherwise even if the App is closed, the timer will still run in the background, eventually causing errors or system freezing.

Best Practice:

Override the `on_close` method:

```
def on_deinit(self):
    if self.timer:
        self.timer.deinit()
```

3.2 Large Image Caches

If you loaded huge images into memory (`f.read()`), it's recommended to manually call garbage collection when exiting:

```
import gc

def on_deinit(self):
    self.big_image_data = None
    gc.collect() # Force memory reclamation
```